

LLD Practice Platform

README + AI Usage

A small practice platform for Low-Level Design problems. The main idea is to make LLD practice more useful by letting a learner submit a design and get feedback on it.

Practice → Submit → Get Feedback → Review → Try Again

What the platform does

A learner can choose an LLD problem, create an attempt, define classes and responsibilities, add relationships, mention patterns used, add an optional code stub, save a draft, submit it for evaluation, review feedback, and try the problem again.

The current problem set includes Parking Lot, Vending Machine, and Elevator System.

Feedback

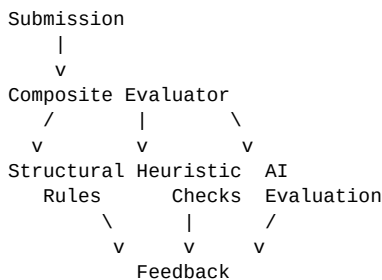
Structural Checks — High confidence: rule-based checks for things that can be checked directly.

Worth Considering — Medium confidence: heuristic checks that point out things the learner may want to review. These are suggestions, not proof that the design is wrong.

AI Insights — Contextual: Gemini is used for feedback that needs more context, such as responsibilities, coupling and cohesion, requirement coverage, extensibility, trade-offs, and design patterns.

This separation is intentional because an LLD problem can have more than one good solution. The platform should help the learner think about their design rather than force one expected answer.

How evaluation works



Deterministic feedback is generated first and is also given to the LLM as context. If AI evaluation is unavailable, the rule-based, heuristic, and requirement-coverage feedback is still kept and the result is marked **llmAvailable: false**.

Main design

The main parts are Problem, Attempt, Submission, Feedback, and Learner. An attempt follows a simple lifecycle: **draft** → **submitted** → **evaluating** → **evaluated**. A **failed** status is available when the complete evaluation pipeline cannot finish.

The evaluation layer is separated into RuleEvaluator, LLMEvaluator, and CompositeEvaluator. This keeps evaluation logic separate from the attempt flow and makes it easier to add another evaluator later.

Submission format

```
classes
  ■■■ name
  ■■■ responsibilities
  ■■■ relationships

patternsUsed

codeStub (optional)
```

I chose this instead of only accepting free-form text because the structure gives the evaluator something concrete to work with while still allowing learners to create different designs.

Requirement coverage

Each problem requirement is checked against the submitted classes, responsibilities, and relationships. The result records the requirement, whether there is evidence of coverage, and which classes contributed to it. This is intentionally a heuristic check rather than a perfect semantic judge.

Tech Stack

Frontend: React, JavaScript, CSS

Backend: Node.js, Express.js, MongoDB, Mongoose

AI: Google Gemini, @google/genai

Testing: Jest, Supertest, MongoDB Memory Server

Running locally

Backend

```
cd backend
npm install
# create .env using .env.example
npm run dev
```

Configure the required environment variables, including the MongoDB connection string and Gemini API key.

Frontend

```
cd frontend
npm install
npm run dev
```

Testing

```
cd backend
npm test

cd frontend
npm test -- --run

npm run build
```

Some decisions I made

Structured submission: I used a structured submission instead of only free-form text so that some parts of evaluation can be deterministic while still allowing different designs.

Layered evaluation: I did not want the LLM to be the only evaluator. Rule-based checks handle objective observations, heuristics provide additional suggestions, and the LLM handles feedback that needs more context.

Multiple valid designs: Expected concepts are treated as suggestions rather than required answers because an LLD problem can have multiple reasonable solutions.

Simple architecture: The application is kept as a monolith for the MVP. Microservices, queues, and distributed components would add complexity without improving the main practice flow.

Limitations

Requirement coverage is heuristic. AI feedback can be imperfect or unavailable. Submitted code is not compiled or executed. Authentication is not included, and the problem set is intentionally small.

Future improvements

I would mainly focus on improving feedback quality: better requirement-to-design mapping, comparing different attempts, diagram-based submissions, more LLD problems, and more consistent AI feedback.

AI Usage

AI was used during this assignment mainly as a development and review aid. I did not use it as a replacement for making the design decisions myself.

1. Submission Format

I considered different ways for a learner to submit an LLD solution. A completely free-form answer would be easy for the learner but difficult to evaluate consistently. Executing arbitrary code would also make the MVP much bigger than needed.

I decided to use a structured submission containing classes, responsibilities, relationships and patterns, with an optional code stub. This gives the evaluator concrete information while still allowing different solutions.

2. Using More Than Just an LLM

One option was to send the complete submission to an LLM and use its response as the evaluation. I did not want to depend on that for everything.

I ended up separating the evaluation into structural checks, heuristic checks and AI feedback. This makes simple checks predictable and leaves the more subjective parts to the LLM.

3. Expected Concepts Are Not Answer Keys

I looked at how problem-specific concepts could be used during evaluation, but decided not to treat them as mandatory answers. An LLD problem can have multiple reasonable designs, so forcing a particular pattern or class would make the feedback less useful.

They are therefore used as signals for heuristic feedback rather than as a fixed solution.

4. Handling AI Failure

I considered making the whole evaluation fail if the Gemini request failed. I rejected that approach because the basic feedback should still be useful when an external AI service is temporarily unavailable.

The current evaluator keeps the structural, heuristic and requirement-coverage feedback and marks **llmAvailable** as false when the LLM part cannot be completed. A complete deterministic/evaluation pipeline failure is handled separately.

5. Keeping the Architecture Simple

During the design process, I considered whether patterns such as a full State Pattern or more infrastructure around evaluation were necessary. For the current scope, I chose simpler solutions.

Attempt status changes are controlled through an explicit transition map, and the application remains a monolith. I felt this was more appropriate for a two-day assignment because the extra infrastructure would add complexity without improving the main practice flow.

What AI helped with

AI was also useful for discussing alternative designs, reviewing implementation decisions, finding bugs during development, suggesting test cases, checking edge cases, and helping write and refine some implementation code.

I reviewed these suggestions and changed or rejected them when they did not fit the actual requirements or the scope of the project. The final implementation was kept aligned with the assignment rather than blindly following AI suggestions.